Developer Assignment: The "Vibe Search" Engine

Scenario:
You are a developer at "StreamFlix," a rapidly growing movie streaming platform. The platform has thousands of movies, but engagement is dropping. Why? Because users are overwhelmed. They spend more time scrolling than watching.

The Problem:
Standard filters like "Action," "Comedy," or "1990s" are too rigid.
A user might want "a sci-fi movie that feels like Blade Runner but is safe for kids" or "a romantic movie that won't make me cry." Current keyword search returns zero results for these complex, natural language queries.

The Challenge:

- You need to build a search interface that "understands" the user's intent.
- It must interpret vague requests (e.g., "fast-paced," "gritty," "heartwarming").
- It must match these requests against a dataset of movies that only has standard metadata (Title, Genre, Plot Summary, Rating).
- It must prioritize accuracy—showing a horror movie to someone asking for "relaxing" is a critical failure.

The Risk:
If the search engine returns irrelevant or offensive results, users will lose trust and cancel their subscriptions. The search needs to be "smart" enough to explain why it picked a movie (e.g., "Recommended because it features a futuristic setting and low violence").

Your Task:

Build a "Vibe Search" feature using React.js and the Gemini API. The user should be able to type a sentence describing exactly what they are in the mood for, and the app should intelligently filter and display the perfect movie matches.

Technical Requirements:

- **Tech Stack:** React.js (Preferred).
  Note: If you are not comfortable with React, you may use Vanilla JS, HTML, and CSS.
- AI Integration: You must use the Google Gemini API.

- Data: You will need to create a dummy JSON dataset of 15-20 movies. Each movie object should have: id, title, genre, release_year, rating (e.g., PG-13), and a short_plot_summary.

Feature Breakdown:

The "Magic Search" Bar: An input field where the user describes their mood (e.g., "I want an 80s adventure movie involving treasure hunting").

Gemini Intelligence:

- **Option A (Filter Logic):** Pass the user's query to Gemini and ask it to convert the text into structured JSON filters (e.g., { "genre": "Adventure", "min_year": 1980, "max_year": 1989, "keywords": ["treasure"] }) to filter your list.
- **Option B (Semantic Match):** Pass the user's query AND your movie list (or a subset) to Gemini and ask it to return the IDs of the top 3 best matches based on the plot summaries.

The Results Grid:
Display the matched movies as cards (Title, Poster placeholder, Year).

The "Why This?" Badge:
For each result, display a one-line AI-generated explanation of why it fits the query (e.g., "Matches your request for 'treasure hunting' and '80s style'").

Empty State:
If no movies match the vibe, show a friendly AI-generated message suggesting what to search for instead.

Things to Keep in Mind:
- Latency: AI API calls are asynchronous. Show a "Scanning library..." loading state or skeleton screen so the user knows something is happening.
- Safety: If a user searches for something inappropriate, the AI should refuse to return results or handle it gracefully.
- UI/UX: The interface should feel like a premium streaming app (Dark mode, clean typography, hover effects on movie cards).

Submission:
Compile all your work into a public GitHub repository.
Deadline: 15 March 2026

Deliverables:
- Source Code: Clean, well-structured code.

- Live Demo or Video: A deployed link (Vercel/Netlify) OR a screen recording showing the search in action.
- Technical Write-up (README.md):
    1. How you designed your prompt to get consistent JSON data from Gemini.
    2. Challenges you faced with "hallucinations" (AI making things up) and how you solved them.
    3. Prompt Logic: A screenshot or code snippet of the specific system instructions you sent to the Gemini API.

Evaluation Criteria:

- Gemini Integration: Does the app truly understand natural language? Can it handle complex queries like "Action movies that aren't violent"?
- Prompt Engineering: How robust is your prompt? Does it reliably return data in a format your code can parse (like JSON)?
- User Experience (UX): Is the transition from "Searching" to "Results" smooth? Does the design look like a modern streaming platform?
- Code Quality: Is the logic separated from the UI? Are API keys handled securely (not hardcoded in the frontend for production, though env variables are fine for this demo)?
- Error Handling: What happens if the API rate limits or fails?

Resources:
Google Gemini API Key: https://aistudio.google.com/
React Tutorial: https://react.dev/learn
TMDB API (Optional): You can use this for real movie data/posters if you don't want to make a dummy JSON file: https://developer.themoviedb.org/docs